说明文档

项目已经部署到Streamlit Cloud [AI 数据可视化助手(这是个链接)]

说明文档

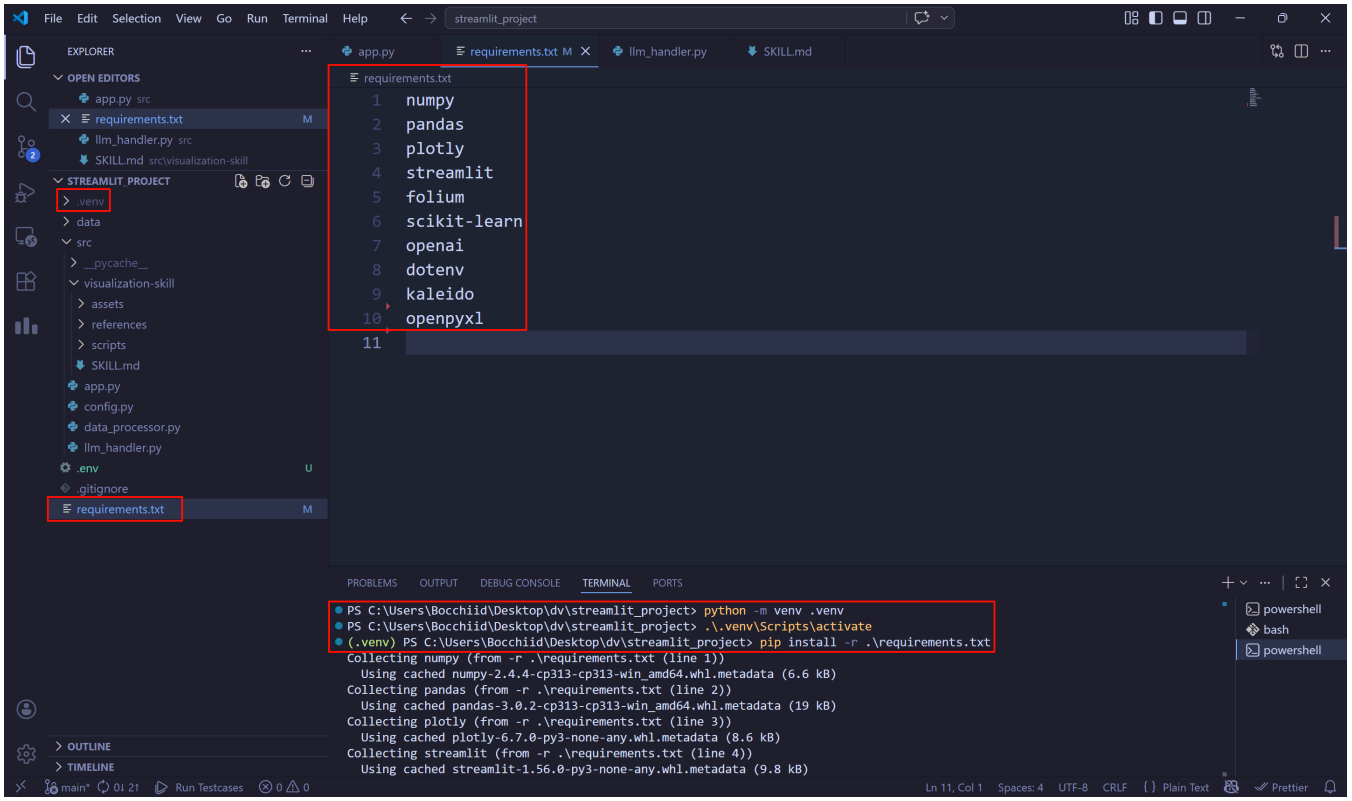
- 项目已经部署到Streamlit Cloud AI 数据可视化助手(这是个链接)
- 开发思路
- 核心功能实现
  - (0) 环境准备
  - (1) 数据处理
    - 1.1 数据上传
    - 1.2 基本数据处理
    - 1.3 数据处理展示(使用slider调节预览的行数, 使用st.session_state来防止每次slider改变时都弹出'数据清洗成功'的消息)
  - (2) 自然语言交互(接入LLM)
    - 2.1 API客户端创建(创建LLMHandler类, 使用openai同步客户端)
    - 2.2 读取Skill(用with open读取skill后注入系统提示词并附上sample_df)
    - 2.3 分离代码块和可视化报告(使用正则匹配)
    - 2.4 用户请求处理部分(调用API方法, 注入历史聊天记录(用于多轮迭代), 降低随机性(temperature), 提高代码执行的成功率)
    - 2.5 初始化LLM(利用st.session_state确保只会初始化一次 提高代码效率)
    - 2.6 用户提示词输入区(使用st.chat_input)
    - 2.7 LLM请求调用(调用st.session_state_llm.chat_for_visualization)
    - 2.8 LLM响应渲染(使用exec执行LLM响应中的代码块, 使用st.info渲染LLM响应中的可视化报告的部分)
  - (3) Skill设计(详看Skill设计说明书)
  - (4) 体验功能优化
- 遇到的问题及解决方法
- 未来可改进的方向
- 总结

开发思路

本项目基于**Streamlit**框架构建, 旨在降低数据可视化的门槛。核心逻辑是将**Pandas**的数据处理能力与**DeepSeek API**的逻辑推理能力结合。通过自定义的`visualization skill`(五段式逻辑), 将LLM定位为“代码生成器”与“数据分析师”的双重角色, 实现从上传数据到获取报告的一键式体验。

核心功能实现

(0) 环境准备



(1) 数据处理

1.1 数据上传

```
# 数据文件上传
    st.sidebar.header("数据集管理")
    uploaded_file = st.sidebar.file_uploader("上传CSV或Excel文件", type=["csv", "xlsx"])
```

1.2 基本数据处理

1. 数据加载(创建DataProcessor类, 利用pandas读取csv, xlsx)

```
class DataProcessor:
    def __init__(self):
        self.df = None

    '''加载CSV或Excel数据文件'''
    def load_data(self, file):
        try:
            if file.name.endswith('.csv'):
                self.df = pd.read_csv(file)
            elif file.name.endswith('.xlsx'):
                self.df = pd.read_excel(file)

            if len(self.df) > 10000:
                st.warning("数据量超过10000行，可能会影响性能。")
            elif len(self.df.columns) > 20:
                st.warning("字段数量超过20列，可能会影响性能。")
            return self.df
        except Exception as e:
```

```
            st.error(f"加载文件失败: {e}")
            return None
```

2. 获取数据集信息(使用pandas方法)

```
'''获取数据维度、字段类型和缺失值统计'''
    def get_basic_info(self):
        if self.df is not None:
            info = {
                "shape": self.df.shape,
                "dtypes": self.df.dtypes,
                "missing_values": self.df.isnull().sum()
            }
            return info
        return None
```

3. 基本数据清洗(数值型用平均值, 字符型用Unknown处理)

```
'''完成基础数据清洗(处理缺失值、异常值)'''
    def basic_data_clean(self):
        if self.df is not None:
            # 数值型填补平均值, 字符型填补'Unknown'
            for col in self.df.columns:
                if self.df[col].dtype in ['int64', 'float64']:
                    self.df[col] = self.df[col].fillna(self.df[col].mean())
                else:
                    self.df[col] = self.df[col].fillna('Unknown')
            return self.df
```

4. 获取前n行数据(增加防越界处理)

```
'''获取前N行数据'''
    def get_n_rows(self, n=10): # 默认预览前10行
        if self.df is not None:
            return self.df.head(min(n, len(self.df))) # 防止n超过数据行数
        return None
```

1.3 数据处理展示(使用slider调节预览的行数, 使用st.session_state来防止每次slider改变时都弹出'数据清洗成功'的消息)

```
# 初始化数据处理模块
    processor = DataProcessor()

# 加载数据
        df = processor.load_data(uploaded_file)

        if df is not None:
            # 显示数据集预览
            st.subheader("数据集预览")

            # 获取raw数据集基本信息
```

```python
            data_info = processor.get_basic_info()

            # 数据清洗
            with st.spinner("正在进行基础数据清洗..."):
                if st.session_state.get(uploaded_file.name) is None:
                    df = processor.basic_data_clean()
                    st.session_state[uploaded_file.name] = True
                    st.toast("数据清洗完成！", icon="✅")
                else:
                    df = processor.basic_data_clean()


            # 展示数据集基本信息
            col1, col2, col3 = st.columns(3)
            col1.metric("样本总量 (行)", f"{data_info['shape'][0]}")
            col2.metric("特征数量 (列)", f"{data_info['shape'][1]}")
            col3.metric("总缺失值", f"{data_info['missing_values'].sum()}")

            st.markdown("### 📋 字段详细信息")

            detail_df = pd.DataFrame({
                "字段名称": data_info['dtypes'].index,
                "数据类型": data_info['dtypes'].values.astype(str),
                "缺失值数量": data_info['missing_values'].values
            })

            st.dataframe(detail_df, use_container_width=True, hide_index=True)

            # N行预览设置
            n_preview = st.sidebar.slider("预览前N行数据", min_value=0, max_value=data_info['shape'][0], value=10, step=1, disabled=st.session_state.is_generating)

            st.markdown("### 🔍 数据预览")
            st.dataframe(processor.get_n_rows(n_preview), use_container_width=True)
```

(2) 自然语言交互(接入LLM)

2.1 API客户端创建(创建LLMHandler类, 使用openai同步客户端)

```python
class LLMHandler:
    '''LLM Constructor, 初始化LLM同步客户端'''
    def __init__(self, api_key: str, base_url: str, model: str):
        # 这里只是简单的接入了API的同步客户端, 后期可改成异步客户端, 提高并发体验
        self.client = openai.OpenAI(api_key=api_key, base_url=base_url)
        self.model = model
```

2.2 读取Skill(用with open读取skill后注入系统提示词并附上sample_df)

```
'''读取visualization-skill'''
    def _get_system_prompt(self, df_schema: str = ""):
        skill_dir = os.path.join(os.path.dirname(__file__), "visualization-skill")
        skill_content = ""

        # 1. 加载SKILL.md
        skill_path = os.path.join(skill_dir, "SKILL.md")
        if os.path.exists(skill_path):
            with open(skill_path, "r", encoding="utf-8") as f:
                skill_content += f.read() + "\n"

        # 2. 注入数据集
        if df_schema:
            skill_content += f"\n## Dataset Context\n{df_schema}\n"

        return skill_content
```

2.3 分离代码块和可视化报告(使用正则匹配)

```
'''提取并分离代码块和可视化报告(正则匹配)'''
    def _extract_code_from_response(self, content: str):
        # 匹配Python代码块
        code_match = re.search(r"```python\n(.*?)```", content, re.DOTALL)
        code = code_match.group(1) if code_match else None

        # 提取代码块之外的所有文字
        interpretation = re.sub(r"```python.*?```", "", content, flags=re.DOTALL).strip()

        return code, interpretation
```

2.4 用户请求处理部分(调用API方法, 注入历史聊天记录(用于多轮迭代), 降低随机性(temperature), 提高代码执行的成功率)

```
def chat_for_visualization(self, user_query: str, data_info, sample_df, history: list = None):
        # 精简Schema以提高速度
        df_schema = (
            f"Columns: {list(data_info['dtypes'].index)}\n"
            f"Dtypes: {data_info['dtypes'].to_dict()}\n"
            f"Sample Preview:\n{sample_df.to_dict()}"
        )

        system_instructions = self._get_system_prompt(df_schema)

        messages = [{"role": "system", "content": system_instructions}]
        if history:
            messages.extend(history)
        messages.append({"role": "user", "content": user_query})
```

```python

        try:
            response = self.client.chat.completions.create(
                model=self.model,
                messages=messages,
                temperature=0.2,
            )
            full_content = response.choices[0].message.content
            code, interpretation = self._extract_code_from_response(full_content)

            return {
                "code": code,
                "interpretation": interpretation,
                "raw_response": full_content
            }
        except Exception as e:
            st.error(f"LLM 调用异常: {str(e)}")
            return None
```

2.5 初始化LLM(利用st.session_state确保只会初始化一次 提高代码效率)

```python
if "llm" not in st.session_state:
        st.session_state.llm = LLMHandler(
            api_key=DEEPSEEK_API_KEY,
            base_url=DEEPSEEK_BASE_URL,
            model=DEEPSEEK_MODEL
        )
```

2.6 用户提示词输入区(使用st.chat_input)

```python
# 自然语言交互
            st.subheader("🤖 AI智能图表生成")

            # 初始化状态：增加用于存放“当前显示的图表结果”的容器
            if "messages" not in st.session_state:
                st.session_state.messages = []
            if "last_viz" not in st.session_state:
                st.session_state.last_viz = None # 存储最新的{code, interpretation}

            # 用户输入指令
            if prompt := st.chat_input("请输入绘图指令... 如: 生成价格与时间的关系图", disabled=st.session_state.is_generating):
                st.session_state.is_generating = True
                st.session_state.current_prompt = prompt
                st.rerun()
```

2.7 LLM请求调用(调用st.session_state_llm.chat_for_visualization)

```
if st.session_state.is_generating:

                current_prompt = st.session_state.get("current_prompt")

                if current_prompt:
                    with st.spinner("AI 正在分析并绘图..."):
                        try:
                            # 使用processor获取精简后的元数据和采样
                            current_data_info = processor.get_basic_info()
                            sample_df = processor.get_n_rows(5)

                            result = st.session_state.llm.chat_for_visualization(
                                current_prompt,
                                data_info=current_data_info,
                                sample_df=sample_df,
                                history=st.session_state.messages[-5:]
                            )
                            if result and result["code"]:
                                # [关键修改]将结果存入session_state, 防止Slider刷新导致消失
                                st.session_state.last_viz = result
                                st.session_state.messages.append({"role": "user", "content": current_prompt})
                                st.session_state.messages.append({"role": "assistant", "content": result["raw_response"]})
                        finally:
                            st.session_state.is_generating = False
                            st.session_state.current_prompt = None
                            st.rerun()
```

2.8 LLM响应渲染(使用exec执行LLM响应中的代码块, 使用st.info渲染LLM响应中的可视化报告的部分)

```
# 渲染区: 放在if prompt之外, 确保每次重跑都能显示
            if st.session_state.last_viz:
                viz = st.session_state.last_viz

                with st.container():
                    try:
                        exec_scope = {"df": df, "px": px, "go": go, "fig": None}
                        exec(viz["code"], globals(), exec_scope)
                        fig = exec_scope.get("fig")

                        if fig:
                            st.markdown("---")
                            # 使用固定key配合session_state里的信息
                            st.plotly_chart(fig, use_container_width=True, key="persistent_chart")

                            st.markdown("#### 📊 可视化解读报告")
                            st.info(viz["interpretation"])
```

```python

                            # 文字报告下载
                            st.markdown("---")

                            # 构造下载内容
                            download_content = (
                                f"数据分析报告\n"
                                f"对应文件: {uploaded_file.name}\n"
                                f"生成时间: {pd.Timestamp.now().strftime('%Y-%m-%d %H:%M:%S')}\n"
                                f"{'='*30}\n\n"
                                f"{viz['interpretation']}"
                            )

                            # 提供TXT下载按钮
                            st.download_button(
                                label="📄 仅下载文字解读报告 (.txt)",
                                data=download_content,
                                file_name=f"分析报告_{uploaded_file.name.split('.')[0]}.txt",
                                mime="text/plain",
                                disabled=st.session_state.is_generating
                            )
                    except Exception as e:
                        st.error(f"图表渲染出错: {e}")
```

(3) Skill设计(详看Skill设计说明书)

```markdown
---
name: visualization-skill
description: "使用Plotly进行数据可视化, 执行5段式skill逻辑, 生成图表并生成文字解读报告"
---

# Visualization Skill(5-Stage)

## 1. 数据深度理解(Data Understanding)

- **字段理解**: 强制检查`df.empty`及列名。提取数值列(`numeric_cols`)和分类列(`categorical_cols`), 识别字段的作用, 为后续智能图表选择提供基础。
- **数据分析**: 统计关键字段的有效值区间与数据分布, 评估字段的展示价值(如: 识别是否为常量列或无效ID列), 为坐标轴映射提供决策支持。
- **返回摘要**: 将数据集的维度信息(shape)、特征关联预览及绘图约束封装为data_summary字典, 构建智能体生成可视化代码的底层上下文。

## 2. 智能图表选择(Chart Selecting)

- **指令解析**: 从`user_input_prompt`中提取关键词(如"趋势"->line, "占比"->pie)选取最适图表类型。
- **降级配置**: 若字段不匹配, 依据`data_summary`强制选取数值列为Y轴, 时间或分类列为X轴, 确保生成的图表合理且程序不中断。
- **元数据生成**: 包含标题、标签等配置的`chart_config`。

```

```
## 3. Plotly图表生成(Plotly Generation)

- **防御性逻辑**: **必须**将绘图代码包裹在`try-except`中。
- **内置处理**: 若绘图失败，必须返回一个带有错误信息注值的`go.Figure()`对象，严禁报崩。
- **样式规范**: 统一使用`plotly.express`或`graph_objects`，严禁使用任何`matplotlib`相关库。
- **样式优化**: **配色遵循ColorBrewer配色原则、合理设置标题/图例/标签、剔除网格线等冗余元素以提升视觉效果，强调可视化逻辑。**

## 4. 文字解读报告(Insights Generation)

- **深度分析**: 基于`fig`包含的数据点分析趋势、极值、异常值。
- **异常修复**: 若第3步失败，此处需输出具体代码修复逻辑，**不需要向用户报告任何跟代码相关的信息，只需报告数据分析的结果即可，范围表示用‘到’连接，报告以分析图表数据的报告为主，不要出现图片更改的解释。**
- **输出定位**: 此报告将作为代码块下方的最终呈现内容供用户阅读。

## 5. 多轮迭代优化(Iteration Workflow)

- **增量修改**: 支持基于`history`修改现有配置(如"颜色改为红色")。
- **校验机制**: 迭代前重新校验新字段在`df`中的存在性。
- **版本控制**: 物理保留旧配置，返回更新后的`config`以供重新绘图。

## 强制约束(Mandatory Constraints)

- **【格式要求】**：必须先输出Python代码块，然后在代码块外部输出环节4的文字解读报告。
- **【库限制】**：严格禁止使用matplotlib，seaborn，plt以及statsmodels。仅允许使用plotly基础库。
- **【核心禁令】**：严禁在px.scatter或其他图表中调用trendline参数(该参数强制依赖 statsmodels)，禁止调用fig.show()。
- **【代码健壮性】**：所有图表逻辑必须仅依赖pandas和plotly的核心功能，禁止使用任何需要额外统计包支持的扩展功能。
- **【迭代要求】**: 迭代修改必须基于用户输入的自然语言指令进行，**迭代重新生成的报告必须以图表数据的分析为主，不要出现图片更改的解释!!!**

```

(4) 体验功能优化

1. 前文提到的使用slider调节预览的行数, 使用st.session_state来防止每次slider改变时都弹出'数据清洗成功'的消息
2. 在st_session_state中增加is_generating字段, 防止LLM生图时被新的请求打断

```
# 在main函数开始处初始化状态
    if "is_generating" not in st.session_state:
        st.session_state.is_generating = False # 标记当前是否正在生成图表，防止重复触发
```

```
# 用户输入指令
            if prompt := st.chat_input("请输入绘图指令... 如: 生成价格与时间的关系图", disabled=st.session_state.is_generating): # 防止LLM的回复被新的请求打断
                st.session_state.is_generating = True
                st.session_state.current_prompt = prompt
                st.rerun()
```

3. 在st_session_state中增加message从而实现历史对话的保存, 以便实现多轮迭代, 增加last_viz, 确保当streamlit有组件发生变动, 导致代码重跑时, 图和报告都能正确渲染, 而不是重跑后导致为空

```
if uploaded_file is not None:
        # 检查当前文件是否与上次记录的文件一致
        if "current_file_name" not in st.session_state or st.session_state.current_file_name != uploaded_file.name:
            st.session_state.current_file_name = uploaded_file.name
            st.session_state.last_viz = None      # 清空上个表的图表结果
            st.session_state.messages = []      # 清空聊天历史(防止AI参考旧表的字段)
```

```
# 初始化状态：增加用于存放“当前显示的图表结果”的容器
            if "messages" not in st.session_state:
                st.session_state.messages = []
            if "last_viz" not in st.session_state:
                st.session_state.last_viz = None # 存储最新的{code, interpretation}
```

```
if st.session_state.is_generating:

                current_prompt = st.session_state.get("current_prompt")

                if current_prompt:
                    with st.spinner("AI 正在分析并绘图..."):
                        try:
                            # 使用processor获取精简后的元数据和采样
                            current_data_info = processor.get_basic_info()
                            sample_df = processor.get_n_rows(5)

                            result = st.session_state.llm.chat_for_visualization(
                                current_prompt,
                                data_info=current_data_info,
                                sample_df=sample_df,
                                history=st.session_state.messages[-5:]
                            )
                            if result and result["code"]:
                                # [关键修改]将结果存入session_state，防止Slider刷新导致消失
                                st.session_state.last_viz = result
                                st.session_state.messages.append({"role": "user", "content": current_prompt})
                                st.session_state.messages.append({"role": "assistant", "content": result["raw_response"]})
                        finally:
                            st.session_state.is_generating = False
                            st.session_state.current_prompt = None
                            st.rerun()

            # 渲染区：放在if prompt之外，确保每次重跑都能显示
            if st.session_state.last_viz:
                viz = st.session_state.last_viz

                with st.container():
```

```python
                    try:
                        exec_scope = {"df": df, "px": px, "go": go, "fig": None}
                        exec(viz["code"], globals(), exec_scope)
                        fig = exec_scope.get("fig")

                        if fig:
                            st.markdown("---")
                            # 使用固定key配合session_state里的信息
                            st.plotly_chart(fig, use_container_width=True, key="persistent_chart")

                            st.markdown("#### 📊 可视化解读报告")
                            st.info(viz["interpretation"])

                            # 文字报告下载
                            st.markdown("---")

                            # 构造下载内容
                            download_content = (
                                f"数据分析报告\n"
                                f"对应文件: {uploaded_file.name}\n"
                                f"生成时间: {pd.Timestamp.now().strftime('%Y-%m-%d %H:%M:%S')}\n"
                                f"{'='*30}\n\n"
                                f"{viz['interpretation']}"
                            )

                            # 提供TXT下载按钮
                            st.download_button(
                                label="📄 仅下载文字解读报告 (.txt)",
                                data=download_content,
                                file_name=f"分析报告_{uploaded_file.name.split('.')[0]}.txt",
                                mime="text/plain",
                                disabled=st.session_state.is_generating
                            )
                    except Exception as e:
                        st.error(f"图表渲染出错: {e}")
```

4. 使用kaleido库, 实现了可视化报告下载的功能

```python
# 渲染区：放在if prompt之外，确保每次重跑都能显示
            if st.session_state.last_viz:
                viz = st.session_state.last_viz

                with st.container():
                    try:
                        exec_scope = {"df": df, "px": px, "go": go, "fig": None}
                        exec(viz["code"], globals(), exec_scope)
                        fig = exec_scope.get("fig")

                        if fig:
                            st.markdown("---")
```

```
                            # 使用固定key配合session_state里的信息
                            st.plotly_chart(fig, use_container_width=True, key="persistent_chart")

                            st.markdown("#### 📊 可视化解读报告")
                            st.info(viz["interpretation"])

                            # 文字报告下载
                            st.markdown("---")

                            # 构造下载内容
                            download_content = (
                                f"数据分析报告\n"
                                f"对应文件: {uploaded_file.name}\n"
                                f"生成时间: {pd.Timestamp.now().strftime('%Y-%m-%d %H:%M:%S')}\n"
                                f"{'='*30}\n\n"
                                f"{viz['interpretation']}"
                            )

                            # 提供TXT下载按钮
                            st.download_button(
                                label="📄 仅下载文字解读报告 (.txt)",
                                data=download_content,
                                file_name=f"分析报告_{uploaded_file.name.split('.')[0]}.txt",
                                mime="text/plain",
                                disabled=st.session_state.is_generating
                            )
                        except Exception as e:
                            st.error(f"图表渲染出错: {e}")
```

5. slider变动时也不会导致图和报告的错误渲染, 因为使用了st.session_state保存了last_viz

遇到的问题及解决方法

1. 文件上传不支持excel, 安装openpyxl库解决
2. slider滑动时, 数据清洗的提示一直弹出, 利用st.session_state保存is_cleaned状态解决
3. 界面刷新导致图表消失
    - **原因**：Streamlit每次操作(如拖动Slider)都会重新运行脚本，导致`exec()`的中间结果丢失
    - **解决**：引入`st.session_state`持久化存储最新的图表结果(`last_viz`)和对话历史
4. LLM偶尔生成不支持的绘图库代码(如 Matplotlib)(主要是生图失败率高且不支持交互 最后定死用plotly了)
    - **解决**：在`SKILL.md`中增加“强制约束”部分，明确禁令并提供标准的Plotly代码模板，通过Prompt规范模型输出

未来可改进的方向

- **图表联动**：实现多图联动, 点击图表A的数据点, 自动过滤并刷新图表B
- **复杂清洗**：增加基于AI的语义化数据清洗(例如识别并统一不同格式的日期字符串)

- **导出增强**：支持直接将图表与解读报告一并导出为PDF报告
- **LLM响应失败兜底**：增加基础的正则过滤用户的自然语言输入, 在LLM响应失败时, 退回到基础的if else选择相应的图表和字段渲染

总结

这次让我深入理解了streamlit的运行逻辑和渲染方式, 我个人的感受 这玩意不好用 一有变动就从上到下重新跑的死逻辑 一点都不灵活

还有因为可视化交互需求 且为了高效 采用了plotly, 但是哪怕Skill增加再强大的限定, 也没法让LLM做到每次生成的代码都能运行成功, 影响体验, 这点是LLM本身的性质决定的, 无解

LLM的引入使得用户的潜在需求能被分析出来并满足, 这点是好事 但不是每次渲染都能成功 可能会陷入多轮迭代的死穴 每次迭代都渲染失败迭代的情况 无解

dv + llm是一个不错的降低dv开发门槛且能更简单的实现更强大功能的组合

但是从体验和生产的角度来说 LLM的不确定性仍旧是最大限制